Modelação e Programação
1º Ano, 2º Semestre

Dulce Mota (mdm@isep.ipp.pt)

Catarina Félix (cfo@isep.ipp.pt)

Git & GitHub

Licenciatura em Engenharia de Sistemas

Introdução

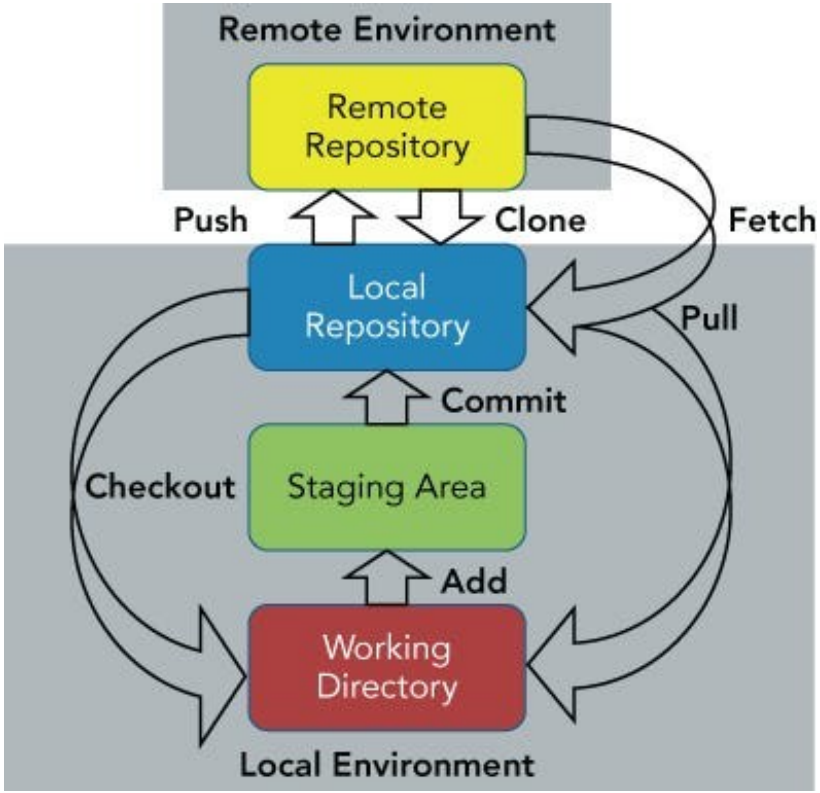
- **Git**: sistema de controlo de versões que mantém registo das alterações efetuadas
  - Duas alternativas:
    - Pela interface do IDE
    - Por linha de comandos: https://git-scm.com/cheat-sheet

- **GitHub**: plataforma cloud que aloja repositórios de código

- **Objetivo**: permitir que várias pessoas trabalhem no mesmo projeto ao mesmo tempo

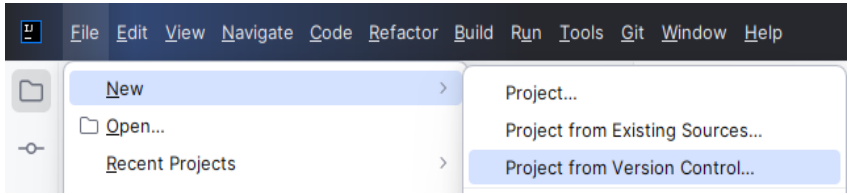
Conceitos

- **Branch**: Ramo de desenvolvimento independente
- **Commit**: guardar alterações com uma descrição adequada
- **Push**: Fazer upload dos commits para o repositório
- **Pull**: Fazer download das alterações a partir do repositório
- **Merge**: Juntar as alterações de dois branches

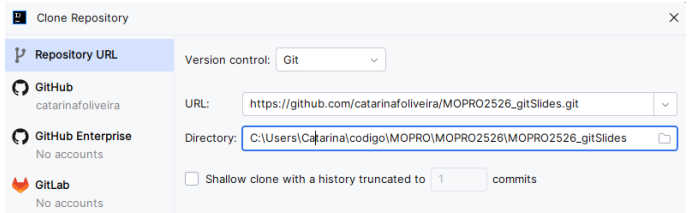


Operações comuns

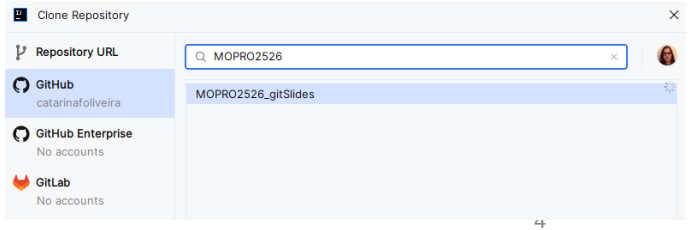
Clone



E depois:

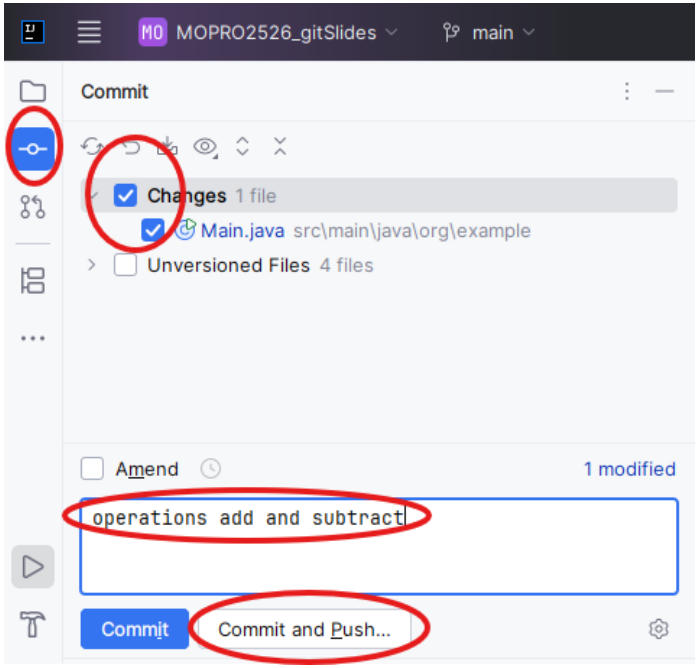


Ou então:



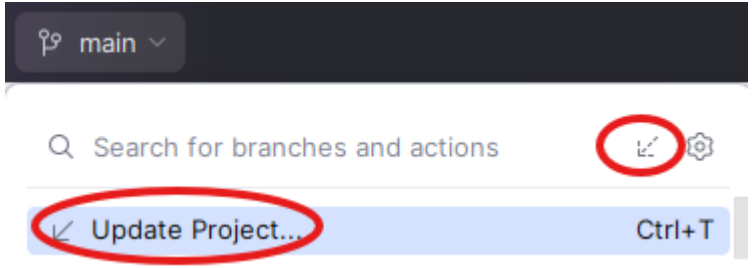
Operações comuns

Commit & Push



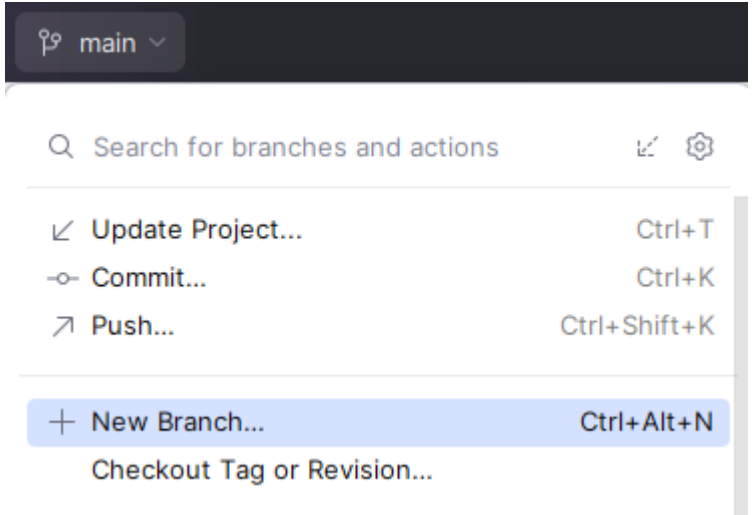
Operações comuns

Pull (da última versão do repositório)



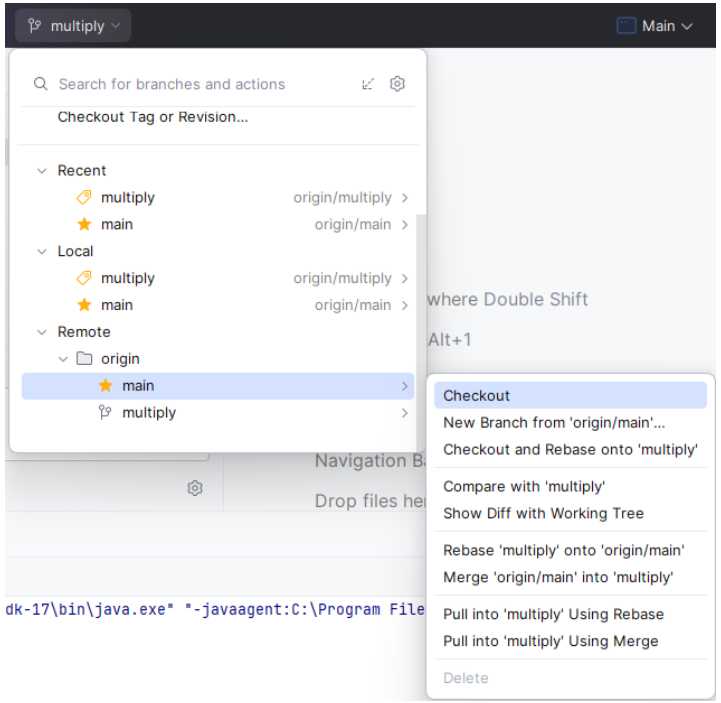
Operações comuns

Criar novo branch



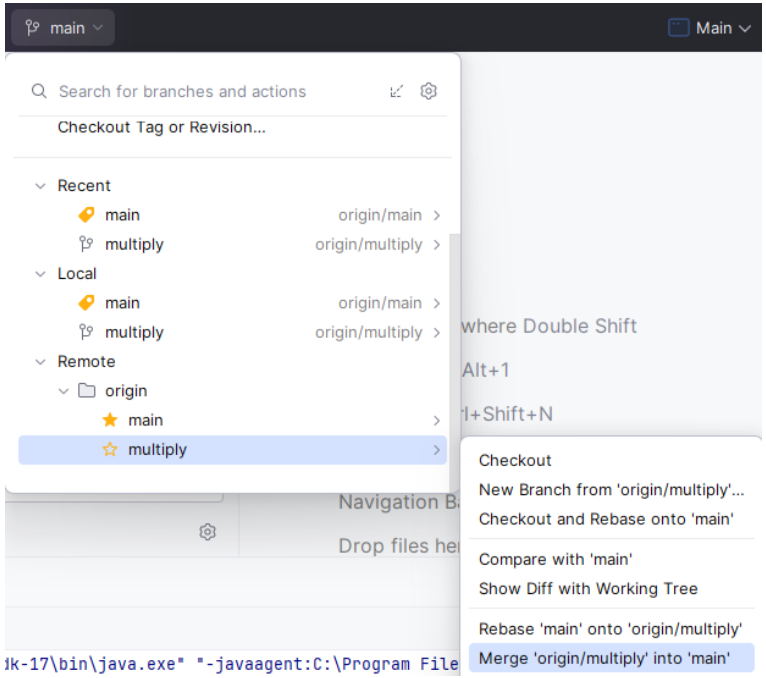
Operações comuns

Voltar para o branch principal



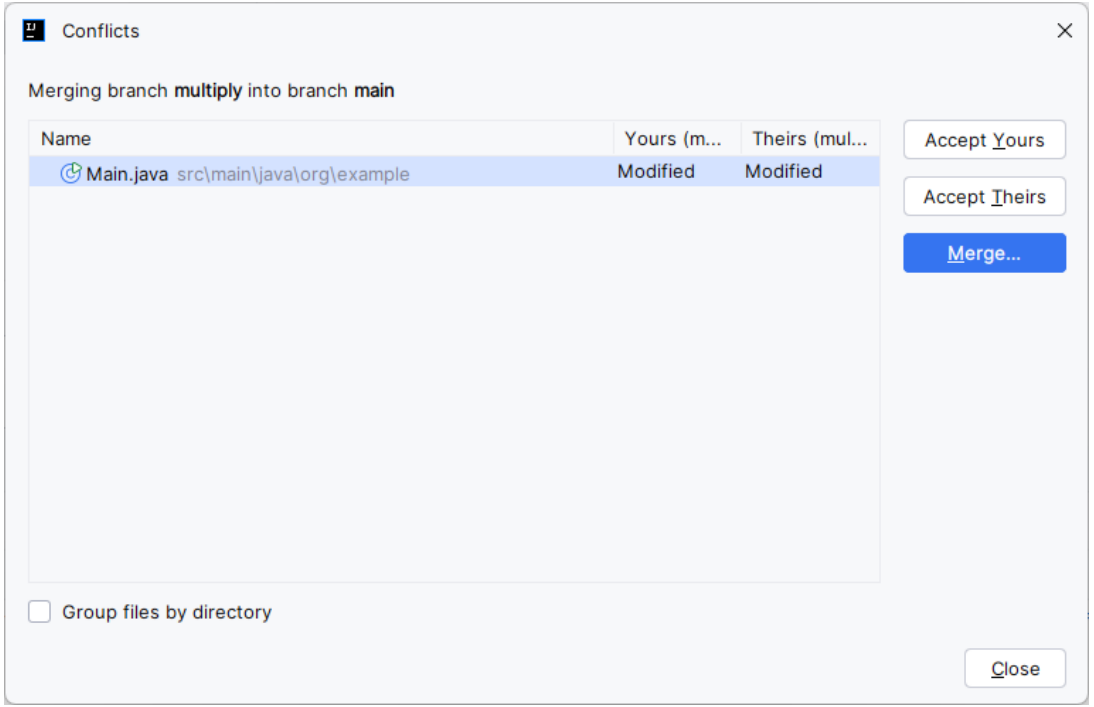
Operações comuns

Fazer merge de um branch para o branch principal



Operações comuns

Após o merge: Notificação de que há conflitos



Clicar em “Merge”

Operações comuns

Conflitos detetados

Alterações em conflito vindas do main:

- Método multiply
- Print do resultado do método multiply

Alterações em conflito vindas do branch:

- Método divide
- Print do resultado do método divide

×: elimina a alteração

>>: faz merge da alteração

Operações comuns

Após merge do método multiply

O código do método multiply foi copiado do bloco da esquerda para o do meio

Operações comuns

Após merge do print do método multiply

O código do print foi copiado do bloco da esquerda para o do meio.
Já não há conflitos vindos do lado do main

Operações comuns

Após merge do método divide

Merge Revisions for C:\Users\Catarina\codigo\MOPRO\MOPRO2526\MOPRO2526_gitSlides\src\main\java\org\example\Main.java

Apply non-conflicting changes: | Do not ignore | Highlight words | No changes. 1 conflict.

Changes from **main** — Show Details

```java
package org.example;


public class Main {
    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b
        return a - b;
    }

    public static int multiply(int a, int b
        return a * b;
    }

    public static void main(String[] args)
        System.out.println("2 + 3 = " + add
        System.out.println("5 - 2 = " + sub
        System.out.println("5 * 2 = " + mul
    }
}
```

Result src\main\java\org\example\Main.java

```java
package org.example;


public class Main {
    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b) {
        return a - b;
    }

    public static int multiply(int a, int b) {
        return a * b;
    }

    public static double divide(int a, int b){
        return a/(double)b;
    }

    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2, 3)
        System.out.println("5 - 2 = " + subtract(
        System.out.println("5 * 2 = " + multiply(
    }
}
```

Changes from **divide** — Show Details

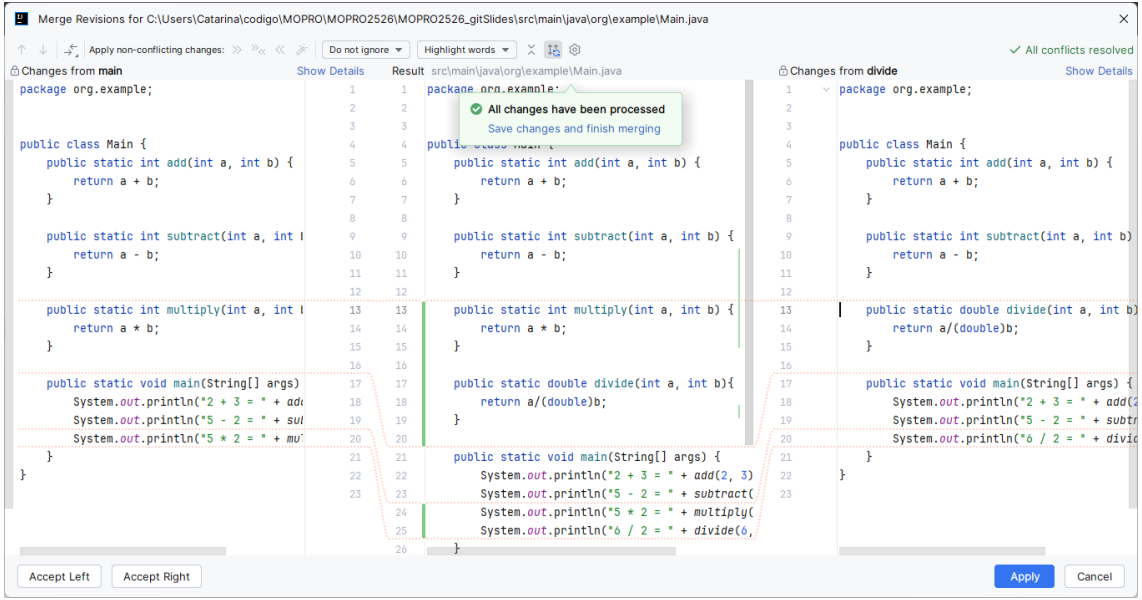
```java
package org.example;


public class Main {
    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b)
        return a - b;
    }

    public static double divide(int a, int b)
        return a/(double)b;
    }

    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2
        System.out.println("5 - 2 = " + subtr
        System.out.println("6 / 2 = " + divid
    }
}
```

Accept Left | Accept Right | Apply | Cancel

O código do método divide foi copiado do bloco da direita para o do meio

Operações comuns

Após merge do print do método divide

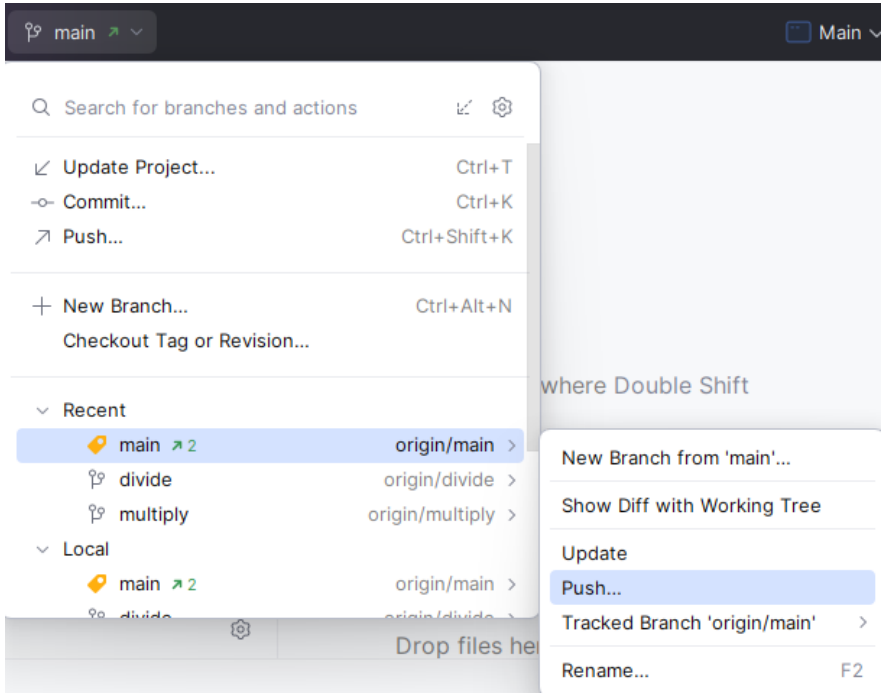


O código do print foi copiado do bloco da direita para o do meio.
Já não há conflitos vindos do lado do branch

Como já não há mais conflitos, aparece a mensagem a verde.
Clicar em “Save changes and finish merging” para terminar

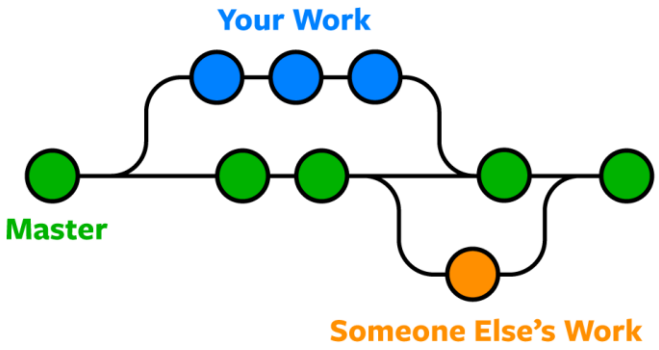
Operações comuns

Fazer Push de um merge



Recomendações

- Fazer Pull **antes de começar a trabalhar**
- Criar um *branch* único para cada *feature* / tarefa
- Fazer Commits **frequentemente** com **descrições adequadas**
- Comunicar com os restantes elementos da equipa para saber o que cada um está a fazer, para facilitar a resolução de conflitos nos *merges*



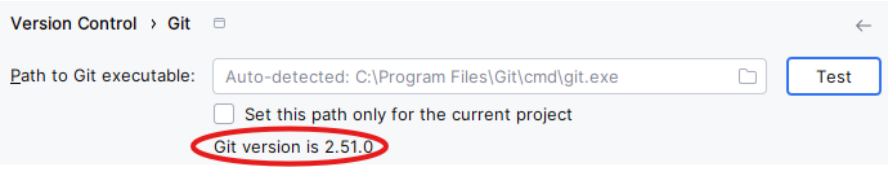
Workflow básico para projetos de grupo

- **Início: Criação/Clone**
  1. Um elemento do grupo cria o repositório e o projeto e adiciona os restantes elementos como colaboradores
  2. Os restantes elementos fazem clone do repositório

- **Diariamente:**
  1. **Fazer Pull da ultima versão antes de começar a trabalhar**
  2. Fazer alterações ao código
  3. Fazer Commits frequentemente, com **mensagens claras**
  4. Fazer Push do trabalho concluído
  5. Resolver conflitos quando surgirem

Configuração Git + IntelliJ

1. (Com o IntelliJ fechado) Instalar o Git: https://git-scm.com/downloads
2. Configurar o Git no IntelliJ:
   - File → Settings → Version Control → Git
   - Assegurar que o caminho para o executável do git esta correto
     - Após clicar em "Test", deve aparecer a versão do Git



   - Por visível a widget do git (apenas necessário uma vez, e só depois de ter um projeto git aberto)
     - View > Appearance > Status Bar Widgets > Git Branch
3. Criar conta no GitHub:
   - Com o email do ISEP
   - Adicionar a conta ao IntelliJ: Settings → Version Control → GitHub → +

Exercício – Calculadora (em grupos de 2)

Aluno A

- Criar novo repositório no GitHub
- Adicionar Aluno B como colaborador
- Criar novo projeto Java Maven
- Na consola, na pasta do projeto: inicializar o repositório, fazer Commit & Push para o repositório:

```
git init
git add .
git commit -m "project creation"
git remote add origin <link do repositorio>
git branch -M main
git push -u origin main
```

Aluno B

- Fazer clone do repositório

Exercício – Calculadora (em grupos de 2)

Aluno A

- Editar a classe Main com o seguinte código:

```
public class Main {
    public static int add(int a, int b) {
        return a + b;
    }

    public static int subtract(int a, int b) {
        return a - b;
    }

    public static void main(String[] args) {
        System.out.println("2 + 3 = " + add(2, 3));
        System.out.println("5 - 2 = " + subtract(5, 2));
    }
}
```

- Verificar que está a correr sem problemas
- Commit (mensagem: operations add and subtract) & Push.

Aluno B

Exercício – Calculadora (em grupos de 2)

Aluno A

- Fazer pull da última versão do repositório.
- Criar novo *branch* chamado multiplication
- Adicionar método multiply à classe
- Adicionar print de uma multiplicação
- Verificar que o projeto corre sem erros
- Fazer Commit (mensagem: operation multiply)
- Fazer Push

Aluno B

- Fazer pull da última versão do repositório.
- Criar novo *branch* chamado division
- Adicionar método divide à classe
- Adicionar print de uma divisão
- Verificar que o projeto corre sem erros
- Fazer Commit (mensagem: operation divide)
- Fazer Push

Exercício – Calculadora (em grupos de 2)

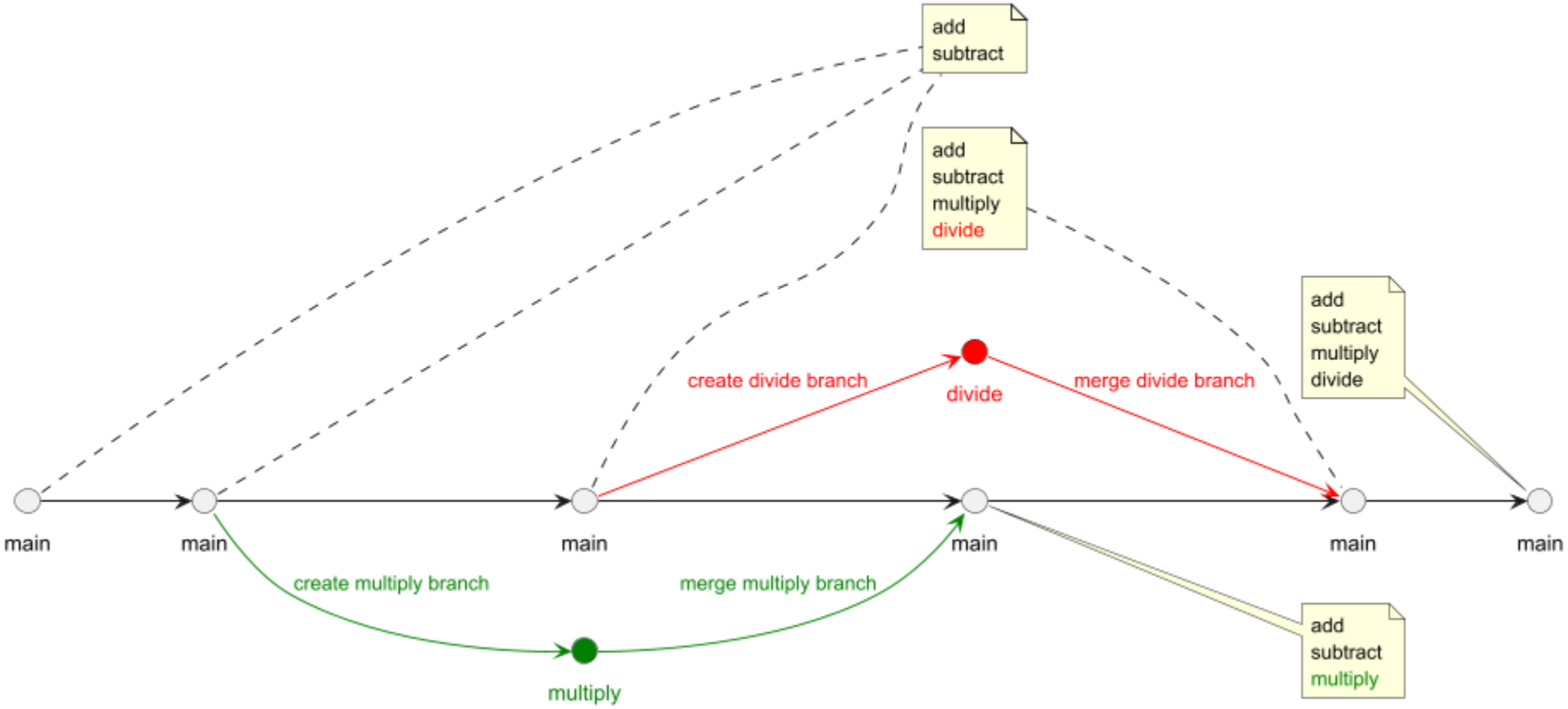
Aluno (primeiro a acabar)

- Voltar para o branch principal
- Fazer pull das alterações mais recentes
- Fazer merge do **seu** branch para o branch principal
- Resolver os conflitos que aparecerem
- Fazer Commit do merge (branch principal)
- Fazer Push do merge para o repositório

- Fazer Pull das alterações mais recentes

Aluno (outro)

- Voltar para o branch principal
- Fazer Pull das alterações mais recentes (já com alterações do colega)
- Fazer merge do **seu** branch para o branch principal
- Resolver os conflitos que aparecerem
- Fazer Commit do merge (branch principal)
- Fazer Push do merge para o repositório

Resultado final esperado: Ambos os alunos têm todas as operações da calculadora funcionais

Árvore do git